



Assignment 7

Aim:- Implement Hospital Emergency Queue to cater services to the patients such as Insert, delete and find highest priority.

Algorithm:

1. Insert operations (insertPatient())

- (a) Create a new Node containing patient's info.

(b) check if queue (head) is empty or if new node's priority is strictly less than the head's node's priority.

- If true: Set $\text{newNode} \rightarrow \text{next} = \text{head}$.

- Update $\text{head} = \text{newNode}$

- Print success message and Exit.

(c) If false, Initialize a pointer $\text{current} = \text{head}$.

(d) Traverse the linked list using a while loop to find the correct position for insertion.

- While $\text{current} \rightarrow \text{next} \neq \text{NULL}$ & $\text{current} \rightarrow \text{next} \rightarrow \text{priority} < \text{newNode} \rightarrow \text{priority}$

- Move current to $\text{current} \rightarrow \text{next}$.

(e) Insert the new Node into its sorted position.

- Set $\text{newNode} \rightarrow \text{next} = \text{current} \rightarrow \text{next}$.

- Set $\text{current} \rightarrow \text{next} = \text{newNode}$.

- Print success message and exit.



2. Find Highest Priority

- (a) check if the queue is empty ($\text{head} == \text{NULL}$)
 - If true: print "The emergency queue is currently empty" and exit .
- (b) Access the head node directly ($\text{head} \rightarrow \text{patient.name}$) and $\text{head} \rightarrow \text{patient.priority}$.
- (c) Display the details of the highest-priority patient.
- (d) exit .

3. delete Highest Priority

- (a) check if the queue is empty ($\text{head} == \text{NULL}$)
 - If true: Print "Emergency queue is empty" and exit .
- (b) Create a temporary pointer temp and assign it to head ($\text{temp} = \text{head}$)
- (c) Update the head pointer to point to next Node
- (d) Print the details of the saved patient stored in temp.
- (e) Free memory allocated for the node pointed by temp ($\text{free}(\text{temp})$).
- (f) Exit .

Conclusion: implemented Hospital Emergency Queue